

How it works, in plain language

Two phones. No SIM, no tower, no internet, no Wi-Fi router. You hold a button and speak in Odia. A second phone across the room says the same thing out loud in Hindi. That is the whole product. This document explains how.

1 · The problem, and the trick

ISRO's problem statement makes one observation and then contradicts itself on purpose:

Voice is the only fair medium. A written message excludes anyone who cannot read. In a disaster that is a large number of people, so the message must be *spoken*.

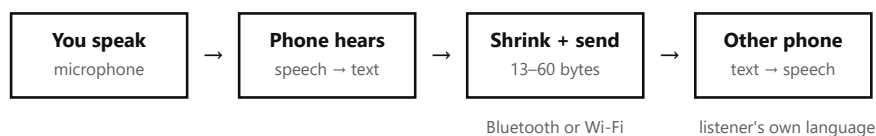
Voice is also the heaviest medium. Audio is far too fat to push through a weak radio link. The link is the one thing you cannot upgrade in a flood.

You cannot solve this with better compression. Opus — the best audio codec there is — stops working below about **6,000 bits per second**. A degraded emergency link gives you around **300**. You are roughly twenty times short before you write any code.

So we stopped trying. **We do not send the voice at all. We send the meaning, and rebuild the voice at the other end.** Everything else in this document follows from that one decision.

2 · The whole journey of one sentence

all four steps happen on the handsets — nothing leaves for a server



3 · Why this is unusual

Almost every voice app you have used sends your audio to a data centre. Ours cannot, and not because we chose to be strict — because in the scenario the app is built for, **there is no data centre reachable**. Every model runs on the handset itself.

That forces three things most apps never have to face: the models must be small enough to sit on a mid-range phone, the phones must find each other with no network to introduce them, and the message must survive a link that drops packets without warning. The next three pages are those three problems.

Turning speech into words

This all happens on the phone in your hand. No audio is uploaded, stored, or kept after the sentence is sent.

4 · Step by step

- ① **You hold the transmit button.**
Push-to-talk, like a walkie-talkie. The microphone opens only while the button is held. That is a privacy decision and a battery decision at once — a phone that listens all day is both creepy and flat by lunchtime.
- ② **A speech model on the phone writes down what you said.**
We use **IndicConformer**, an open speech model trained by AI4Bharat on Indian languages. It runs through **ONNX Runtime**, which is a small engine for running AI models on ordinary devices. One model per language, about **190 MB** each.
- ③ **The phone leans on the words that matter.**
In an emergency the important vocabulary is small and predictable — *injured, trapped, water rising, sector, evacuate*. We give the recogniser a list of those words per language so it prefers them when a sound is ambiguous. Getting "trapped" right matters far more than getting an ordinary word right.
- ④ **You see the text before it goes.**
The screen shows what the phone thinks you said, and how confident it is. If it heard you wrong, you have not yet sent anything. Alerts require a second confirming press.

5 · Why the models are not inside the app

The app itself is **27 MB**. The language models are around **190 MB each**. Putting ten languages inside the installer would make it 2 GB, which nobody can install over a weak connection.

So the app ships empty and the operator installs the one or two languages that unit needs. The files are downloaded **once, during setup, by the phone's ordinary browser** — the app itself never fetches anything. It hands the address to the browser, then imports the finished file from the Downloads folder and checks it with a SHA-256 hash before trusting it.

Why the hash check matters

A file called `model.int8.onnx` could be any of ten languages, and a half-finished download looks exactly like a complete one. The filename tells you nothing. The hash tells you exactly which language it is and whether all the bytes arrived. A pack that fails the check is rejected rather than installed as silence.

6 · What this costs, honestly

Running a speech model on a phone is slower than running it in a data centre, and the accuracy is slightly lower than a cloud service. We accept both, because a cloud service scores zero when the tower is down — and the tower being down is the entire scenario.

Sixty bytes, and three radios

This is the part that makes the project work at all — and the part that is genuinely different from other messaging apps.

7 · The table trick

Both phones carry the **same table of emergency sentences**, written out in all ten languages. When your sentence matches one of them, we do not send the words. We send the **row number**.

One byte, meaning "row 7". The receiving phone looks up row 7 **in its own language** and speaks it. Odia goes in, Hindi comes out — with **no translation model anywhere**. The shared table *is* the translation, and it was written by humans in advance, so it cannot mistranslate under pressure.

If what you said is not in the table, we fall back to sending the actual text, packed tightly. That is bigger, but still tiny next to audio.

~13 B a template alert on the wire	~60 B a free-form sentence, sealed	1,600× smaller than the audio it replaces	43× smaller than Opus at its floor
--	--	---	--

8 · What is actually in those bytes

Each message is wrapped in a small envelope of 10 bytes at the front and 2 at the back. It carries a marker so a receiver can find the start of a message in a noisy stream, the language, a sequence number, who sent it, how many more hops it may travel, and a checksum. The message itself is sealed with **AES-256-GCM** encryption, so a listener without the key learns nothing and cannot alter it without being caught.

There is deliberately **no "to" field**. Every message goes to every unit, exactly like a walkie-talkie. That single decision is why the network needs no setup.

9 · Three radios, used at the same time

ROAD	HOW IT WORKS, AND WHAT IT COSTS
Bluetooth LE broadcast	The main one. The phone puts the message inside a Bluetooth <i>advertisement</i> — the same beacon mechanism a fitness band uses — and every phone nearby with the app open reads it. No pairing, no PIN, no connection, no list of devices . Two phones with the app open are already on the network. Pairing is the most common cause of demo failure, so we removed it.
Wi-Fi broadcast	One phone turns on its hotspot. Others join it. Messages are shouted to everyone on that little network. Still no internet, no router, no data plan — just the phones. This road has the bandwidth to carry real audio when we need a fallback.
Bluetooth Classic (paired)	If two handsets happen to be paired already, we also use that, because it is much faster. It is a bonus, not a requirement.

Every message goes down **all three at once**. If one road is blocked, the others still carry it. Because of that, the same message often arrives two or three times — so each phone remembers what it has already seen and **delivers it once**.

10 · Out of range

If a unit is too far away, any phone that heard the message re-broadcasts it. Each message may be passed on **three times** before it stops, and each phone refuses to pass on a message it has already handled — otherwise three phones would echo each other forever.

The far end, and what is left

11 · Turning the bytes back into a voice

- 1 **Check before trusting.**
The receiving phone verifies the checksum, then the encryption tag. If a single bit was altered, the message is thrown away rather than spoken. It also refuses a message it has already received.
- 2 **Render it in the listener's language.**
If it is a table row, the phone looks it up in *its own* language setting. A Hindi handset shows Hindi, whatever the sender spoke.
- 3 **Speak it aloud.**
A small neural voice model — **Piper**, also running on the phone through ONNX Runtime — turns the text into speech. Alerts are announced at full volume even if the phone is on silent and locked, because that is what the problem statement requires.

12 · What is true today

10 of 10	languages are recognised and displayed
7 of 10	languages are also spoken aloud (see below)
27.1 MiB	the installed app, without language packs
3 ways	the offline claim is checked: no web code in the app, no address is ever looked up, and every packet is addressed to "everyone nearby"
0	servers, accounts, analytics or crash reporting — there is no back end to this product at all

13 · What we have not finished, and say so

Three languages do not speak yet
Tamil, Kannada and Odia recognise and display, but there is **no freely licensed voice** published for them that fits on a phone. We searched every model family our engine supports — 642 published models — and the only free voices that cover those three are far too large for a mid-range handset. We refused the one that fits because its licence forbids commercial use, and hiding that would be worse than the gap. A *table* message still arrives spoken in all ten languages, because the receiving phone does the speaking from its own table.
Gujarati was in that list until this week and no longer is — we found a suitable free voice for it and shipped it.

The speed and accuracy figures are not measured yet
The app times every stage of the journey on the live path, and it will write the result out as a spreadsheet. But it **refuses to produce that file** unless the run meets the conditions we set ourselves: a real release build, a real handset, thirty minutes of continuous use, off charge, and at least a hundred sentences. Until such a run happens, we quote no latency or accuracy number. A figure we cannot reproduce is worth less than admitting we do not have it yet.

14 · The one sentence version

We stopped trying to squeeze the voice down the wire. We send the **meaning** in a few dozen bytes, and let the phone at the far end say it out loud — in that listener's own language, with no server, no SIM and no internet anywhere in the path.